Session – 13

Task-1.

Create a table named Playlists with columns: id, user_id, playlist_name, and total_likes. Insert at least 8 sample rows with different users and playlists, making sure some playlists have the same user_id.

Solution:

```
CREATE TABLE Playlists (
  id INT PRIMARY KEY,
  user_id INT,
  playlist_name VARCHAR(100),
  total_likes INT
);


INSERT INTO Playlists (id, user_id, playlist_name, total_likes)
VALUES
(1, 101, 'Chill Vibes', 500),
(2, 101, 'Workout Hits', 800),
(3, 101, 'Party Mix', 800),
(4, 102, 'Bollywood Hits', 1200),
(5, 102, 'Romantic Songs', 600),
(6, 103, 'Travel Songs', 900),
(7, 103, 'Morning Music', 400),
(8, 104, 'Lo-Fi Beats', 1200);
```

Task-2.

Solution:

```
SELECT
  playlist_name,
  user_id,
  total_likes,
  ROW_NUMBER() OVER (
    ORDER BY total_likes DESC
```

```
  ) AS row_num
FROM Playlists;
```

Write a SQL query using ROW_NUMBER() and the OVER() clause to assign a unique row number to each playlist, ordered by total_likes in descending order.

Task-3.

Solution:

```
SELECT
  playlist_name,
  user_id,
  total_likes,
  RANK() OVER (
    ORDER BY total_likes DESC
  ) AS playlist_rank
FROM Playlists;
```

Use the RANK() function with the OVER() clause to rank all playlists by total_likes, and display the playlist_name, user_id, total_likes, and their rank.

Task-4.

Solution:

```
SELECT
  playlist_name,
  user_id,
  total_likes,
  DENSE_RANK() OVER (
    PARTITION BY user_id
    ORDER BY total_likes DESC
  ) AS dense_rank
FROM Playlists;
```

Write a SQL query using DENSE_RANK() and PARTITION BY user_id to rank each user's playlists by total_likes, showing playlist_name, user_id, total_likes, and dense rank.

Hint: This will show how popular each playlist is within each user's account, similar to how Spotify might rank your top playlists.

Solution:

```
SELECT
  playlist_name,
  user_id,
  total_likes,
  DENSE_RANK() OVER (
    PARTITION BY user_id
    ORDER BY total_likes DESC
  ) AS dense_rank
FROM Playlists;
```

Task-5.

Imagine you want to show the top 2 playlists per user based on total_likes, like Spotify's 'Your Top Playlists' feature. Write a query using a window function to select only the top 2 playlists for each user.

Solution:

```
WITH RankedPlaylists AS (
  SELECT
    playlist_name,
    user_id,
    total_likes,
    ROW_NUMBER() OVER (
      PARTITION BY user_id
      ORDER BY total_likes DESC
    ) AS rn
  FROM Playlists
)
SELECT
  playlist_name,
```

```
    user_id,
    total_likes
FROM RankedPlaylists
WHERE rn <= 2;
```